

SEED: 확장 가능한 분산 서브그래프 열거

Scalable Distributed Subgraph Enumeration

Longbin Lai, Lu Qin, Xuemin Lin, Ying Zhang, Lijun Chang, Shiyu Yang (2016)

후보 폭발을 줄이는 저장 구조·조인 단위·실행 계획·클릭 압축의 결합

입력은 두 그래프, 출력은 모든 match

작은 pattern P를 큰 data graph G에 injective하게 배치한 결과를 전부 찾는다

- | | | |
|----|--------------|---------------------------------------|
| 01 | Data graph G | 큰 undirected·unlabeled graph |
| ↓ | | |
| 02 | Pattern P | 작고 connected한 query graph |
| ↓ | | |
| 03 | Match f | Pattern vertex를 서로 다른 data vertex에 대응 |
| ↓ | | |
| 04 | Enumerate | 조건을 만족하는 모든 mapping을 출력 |

존재 여부가 아니라 모든 결과를 출력하므로 output 자체도 커질 수 있다

Decision과 enumeration은 종료 조건이 다르다

첫 성공에서 멈추는 문제와 모든 성공을 출력하는 문제를 구분해야 한다

구분	질문	종료 조건	주요 부담
Decision	Match가 하나라도 있는가	첫 증거를 찾으면 종료 가능	탐색 난도
Counting	Match가 몇 개인가	정확한 개수 계산	집계 비용
Enumeration	Match가 무엇인가	모든 match 출력 후 종료	후보·중간 결과·출력

SEED가 줄이는 핵심은 최종 정답이 아니라 정답까지 가는 중간 결과다

기본 match의 두 조건에 대칭 제거 조건을 더한다

서로 다른 pattern vertex를 같은 data vertex에 겹치게 놓을 수 없다

조건	검사	실패 예
Conflict freedom	서로 다른 v 는 서로 다른 $f(v)$	$v1$ 과 $v3$ 을 모두 a 에 배치
Structure preservation	Pattern edge가 data edge로 존재	$v1-v2$ 인데 $a-b$ edge가 없음
조건부: Order preservation	대칭 제거용 partial order를 유지	$v1 < v3$ 인데 $f(v3) < f(v1)$

앞의 두 조건은 match 정의이고 세 번째는 대칭 중복을 제거한다

Symmetry breaking은 같은 subgraph의 중복 표현을 막는다

Pattern의 대칭을 partial order로 고정해 대표 mapping 하나만 남긴다

- | | | |
|----|---------------|---------------------------------------|
| 01 | Automorphism | Pattern을 자기 자신에 대응시키는 대칭 |
| ↓ | | |
| 02 | Partial order | 일부 pattern vertex에 $v_i < v_j$ 지정 |
| ↓ | | |
| 03 | Data order | Degree와 ID로 data vertex 전체 순서 정의 |
| ↓ | | |
| 04 | Filter | $v_i < v_j$ 이면 $f(v_i) < f(v_j)$ 만 허용 |

대칭 제거는 후보 폭발 문제를 줄이지만 조인 순서 문제를 대신하지 않는다

High-degree vertex 하나가 후보를 폭발시킨다

Degree가 백만이면 작은 star도 조합 수가 기계 용량을 압도할 수 있다

부분 패턴	중심 degree	후보 규모	왜 생기는가
3-edge star	10^6	$O(10^{18})$	이웃 세 개 조합
2-edge TwinTwig	10^6	$O(10^{12})$	이웃 두 개 조합
Clique unit	같은 중심	Data edge에 따라 감소	이웃 사이 edge도 즉시 검사

TwinTwig는 지수를 낮추지만 후보 생성을 근본적으로 없애지는 않는다

분산 환경에서는 DFS보다 join으로 나누기 쉽다

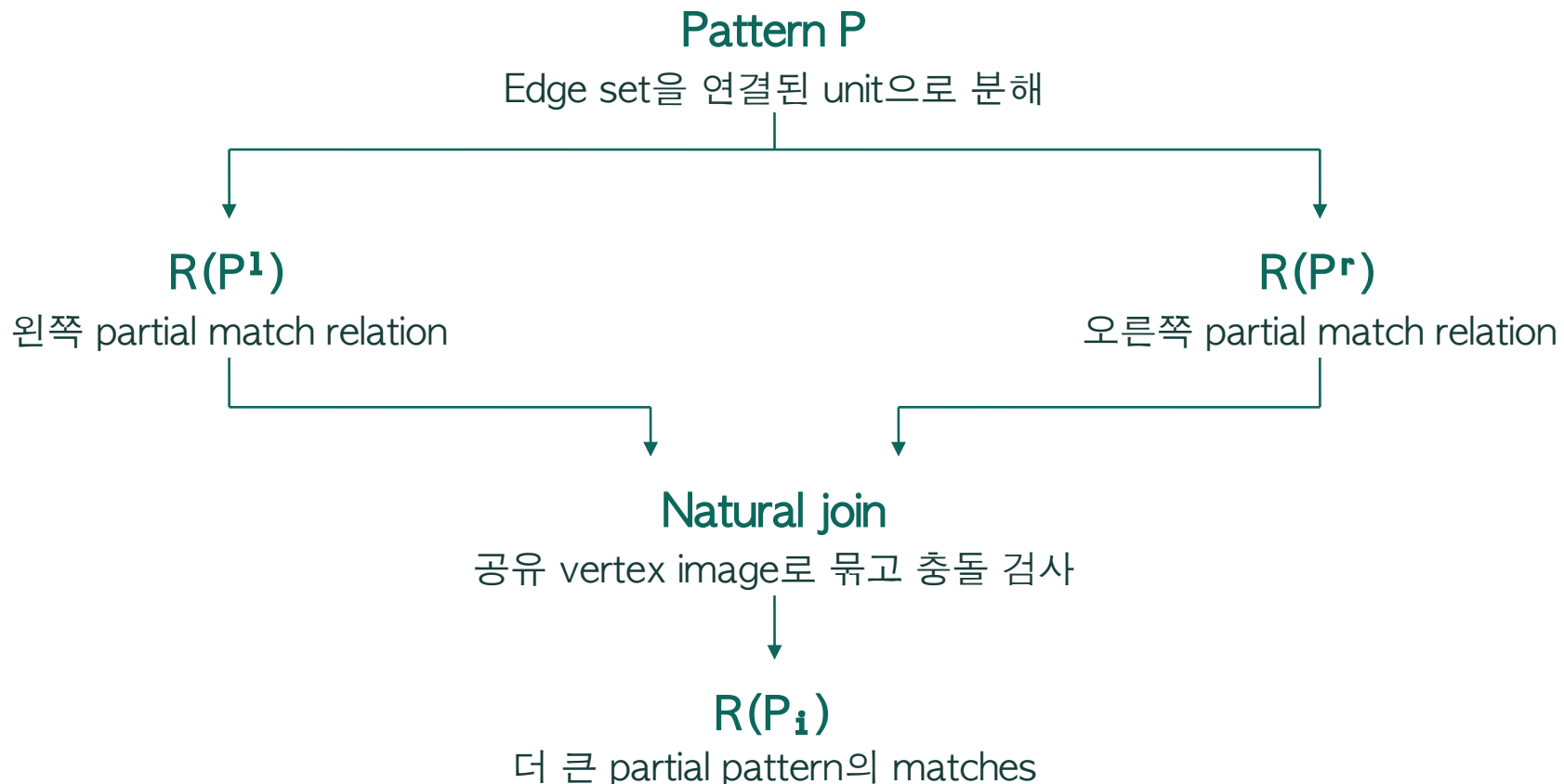
Pattern을 작은 relation으로 만들고 공통 attribute를 key로 병렬 결합한다

- | | | |
|----|-----------------|---|
| 01 | Decompose P | Pattern을 작은 join unit으로 분해 |
| ↓ | | |
| 02 | Enumerate units | 각 unit의 local match relation 생성 |
| ↓ | | |
| 03 | Shuffle by key | 공유 pattern vertex의 data image로 grouping |
| ↓ | | |
| 04 | Join relations | Conflict-free tuple만 결합 |
| ↓ | | |
| 05 | Repeat | 전체 pattern relation까지 확장 |

병렬화가 쉬워지는 대신 중간 relation 크기가 새 병목이 된다

Pattern relation은 join unit을 결합해 계산한다

두 partial pattern이 공유하는 vertex image가 join key가 된다



Reducer는 같은 key라도 tail이 겹치면 버린다

공유 vertex는 같아야 하지만 나머지 vertex image는 서로 달라야 한다

입력	공유 key	Tail 검사	결과
$f=(a,b,c), h=(b,d)$	b	$\{a,c\} \cap \{d\} = \emptyset$	fUh 출력
$f=(a,b,c), h=(b,a)$	b	$\{a,c\} \cap \{a\} = \{a\}$	충돌로 폐기
여러 f와 h	동일 tuple	모든 pair 검사	Conflict-free pair만 출력

Join key 일치만으로는 injective mapping이 보장되지 않는다

TwinTwigJoin의 optimality에는 두 경계가 있다

Instance optimal이라는 말은 star unit과 left-deep search 안에서만 성립한다

경계	TwinTwigJoin	SEED의 확장
Join unit	1-edge 또는 2-edge star	Star와 clique
Join tree	Left-deep만 탐색	Bushy tree 전체 탐색
Graph model	ER 기반 비용	Power-law PR 기반 추정
고차수 vertex	TwinTwig 후보도 $O(d^2)$	Clique로 early filtering 가능

SEED는 같은 제한 공간에서 더 잘 고르는 것이 아니라 탐색 공간을 넓힌다

Left-deep과 bushy는 중간 결과의 모양이 다르다

Bushy plan은 두 개의 선별된 partial relation을 마지막에 합칠 수 있다

구조	각 내부 join	가능한 효과	탐색 비용
Left-deep	한 child가 늘 join unit	한쪽 partial이 계속 성장	허용 plan 후보가 제한됨
Bushy	두 child 모두 partial 가능	작은 결과끼리 먼저 결합	더 큰 search space
관계	Left-deep은 bushy의 일부	최적 bushy cost는 더 나쁠 수 없음	DP가 필요

Bushy의 장점은 round 수보다 거대한 intermediate relation을 피할 선택지다

SEED는 다섯 구성요소를 하나의 실행 계획에 묶는다

각 구성요소는 다른 병목을 겨냥하고 서로의 선택 범위를 넓힌다

01	SCP storage	Star와 clique를 local join unit으로 허용
↓		
02	PR estimate	Power-law degree에서 relation 크기 추정
↓		
03	Bushy DP	더 넓은 join tree 중 최소 추정 cost 선택
↓		
04	Overlap	선별 edge를 unit에 중복 포함
↓		
05	Compression	큰 data clique의 조합을 compact하게 유지

Clique를 쓸 저장 구조만 만들고 plan을 고르지 못하면 전체 이득을 얻기 어렵다

무엇을 local graph에 넣느냐가 join unit을 결정한다

Join unit의 match는 각 local graph에서 독립적으로 만들고 합쳐 완전해야 한다

- Storage $\phi(G)$ 는 data vertex u 마다 connected local graph G_u 를 저장한다
- 모든 local graph의 edge union은 원래 $E(G)$ 와 같아야 한다
- Join unit p 의 local match union은 global $R(p)$ 와 같아야 한다
- Local graph에 없는 edge를 요구하는 pattern은 독립 enumeration이 불가능하다
- 따라서 storage 확대는 unit 확대와 저장-skew 증가를 동시에 만든다

Join unit은 query 문법이 아니라 storage가 보존하는 구조다

ϕ_0 는 중심과 incident edge만 저장한다

TwinTwigJoin의 local graph는 u 와 이웃을 담지만 이웃 사이 edge는 담지 않는다

Local graph G_u^0	포함	누락	가능한 unit
Vertex	u 와 $N(u)$	없음	Star 중심을 u 로 고정
Edge	u 에서 이웃으로 가는 edge	이웃끼리의 edge	1-edge, TwinTwig, star
Triangle	두 incident edge	Closing edge	Clique는 독립 생성 불가

ϕ_0 가 star-only인 이유는 algorithm 이름이 아니라 closing edge 부재다

ϕ_1 는 clique를 보존하지만 skew를 키운다

각 u 의 모든 이웃 사이 data edge를 G_u' 에 복제하면 triangle closing edge를 모두 얻는다

성질	ϕ_1 의 선택	결과
Closing edge	$N(u)$ 내부의 모든 data edge 추가	Star와 clique 보존
Extra edge	기대값이 w_i^2 에 거의 비례	고차수 local graph 팽창
분산 실행	특정 G_u' 에 edge 집중	Worker workload skew
설계 판단	지원 unit은 늘지만 크기 bound가 나빠짐	더 compact한 ϕ_2 필요

모든 triangle edge를 모든 관련 중심에 복제하면 clique 지원 비용이 과해진다

ϕ_2 는 triangle closing edge의 owner를 하나로 정한다

Triangle (u, u', u'') 에서 u 가 두 이웃보다 작을 때만 closing edge를 G_u^2 에 넣는다

01	Triangle 발견	세 data vertex u, u', u'' 가 서로 연결
↓		
02	Order 비교	Degree와 ID로 정의한 $<$ 적용
↓		
03	Minimum 선택	$u < u'$ 이고 $u < u''$ 인 u 가 owner
↓		
04	Edge 추가	Closing edge (u', u'') 를 G_u^2 에만 저장
↓		
05	Local clique	한 local graph에서 clique를 완전 열거

Ordered minimum 규칙은 triangle edge 복제를 줄이면서 clique 보존을 유지한다

손으로 추적하기: ϕ_0 에서는 triangle이 보이지 않는다

Data edge는 a-b, a-c, b-c, a-d이고 order는 $a < b < c < d$ 라고 하자

단계	G_a 에 있는 edge	관찰
원본 data	a-b, a-c, b-c, a-d	{a,b,c}가 triangle
ϕ_0 구성	a-b, a-c, a-d	중심 a의 incident edge만 저장
Triangle query	a-b와 a-c는 확인	b-c가 local graph에 없음
결과	Star 후보는 생성	Clique match는 아직 독립 생성 불가

Global graph에 b-c가 있어도 ϕ_0 의 G_a 만 보면 그 edge를 확인할 수 없다

손으로 추적하기: b-c의 owner는 a다

Closing edge는 triangle의 최소 vertex가 중심인 local graph 한 곳에만 들어간다

후보 owner	최소 조건	Closing edge 저장	Local triangle
a	$a < b, a < c$ 모두 참	G_a^2 에 b-c 추가	a-b, a-c, b-c 완성
b	b < a가 거짓	G_b^2 에 a-c 추가 안 함	이 triangle의 owner 아님
c	c < a와 c < b가 거짓	G_c^2 에 a-b 추가 안 함	이 triangle의 owner 아님

같은 triangle을 세 local graph에 복제하지 않아도 한 곳에서 정확히 열거할 수 있다

ϕ_2 가 보존하는 join unit은 정확히 star와 clique다

더 많은 motif를 지원하지 않는 것은 누락이 아니라 local graph 크기와의 trade-off다

Pattern family	ϕ_2 join unit	이유
Star	가능	중심 incident edge는 모두 존재
Clique	가능	Ordered-min local graph에 closing edge가 모임
임의 motif	일반적으로 불가	필요 edge가 한 local graph에 모인다는 보장 없음
더 넓은 storage	이론상 가능	Local edge 수 bound와 skew가 악화될 수 있음

SCP는 Star-Clique-Preserved이며 모든 pattern-preserved storage가 아니다

비용 함수는 Map·Shuffle·Reduce data volume을 더한다

MapReduce에서는 network I/O와 disk I/O를 만드는 중간 data가 지배적이라고 본다

01	Map M	입력 읽기와 mapper output
↓		
02	Shuffle S	Mapper에서 reducer로 전송
↓		
03	Reduce R	Reducer input과 partial output
↓		
04	Plan cost	각 round의 M+S+R 합산

여기서 M은 map volume이며 앞에서 data edge 수에 쓴 M과 문맥이 다르다

Join unit과 partial pattern은 다른 비용식을 쓴다

Join unit은 local graph에서 새로 만들고 partial relation은 이전 round 결과를 읽는다

입력 종류	비용	항의 의미
Join unit p	$T(p) = \Delta(G) + 3 R(p) $	Local storage scan과 match의 map·shuffle·reduce
Partial Pa	$T(Pa) = 5 R(Pa) $	기존 relation의 read·write·shuffle
Execution E	$C(E) = \text{모든 } T \text{의 합}$	모든 unit과 중간 partial pattern을 합산

Plan 비교에는 각 subpattern relation 크기 $|R(P)|$ 의 추정이 필요하다

PR model은 edge를 확장하며 match 수를 추정한다

Expected degree sequence를 이용해 tree edge와 non-tree edge의 증가율을 다르게 계산한다

01	Start edge	Undirected edge relation 크기를 $2M$ 으로 시작
↓		
02	DFS tree	새 vertex를 붙이는 tree edge를 순서대로 추가
↓		
03	Case 1	Degree moment ratio로 새 이웃 수 γ 추정
↓		
04	Back edge	기존 두 vertex를 잇는 non-tree edge 추가
↓		
05	Case 2	두 degree moment와 정규화 계수 ρ 반영

Heavy-tail degree를 반영하므로 ER model과 다른 join plan을 고를 수 있다

PR 식의 기호: data degree와 pattern degree를 구분한다

w_i 는 data graph의 기대 degree, δ 는 확장 직전 partial pattern의 degree다

기호	정의	역할
w_i	Data vertex i 의 expected degree	PR 모델 입력
ρ	$1 / \sum_i w_i$	$\Pr\{(i,j) \in E\} = w_i w_j \rho$
δ, δ'	확장 전 pattern endpoint degree	Degree moment의 지수
$\mu(\delta)$	$\sum_i w_i^{(\delta+1)} / \sum_i w_i^\delta$	다음 식을 위한 축약 표기
γ	한 edge 확장 시 match-count 계수	현재 PR count에 곱함

μ 는 식을 읽기 위한 축약이며 ρ 자체를 edge probability라 부르지 않는다

새 이웃을 붙일 때와 기존 두 정점을 닫을 때의 계산

시작 edge의 oriented relation은 2M이며 확장할 때마다 해당 γ 를 곱한다

확장	계산식	해석
새 vertex 추가	$\gamma = \mu(\delta)$	δ -degree endpoint에 새 이웃 부착
기존 두 vertex 연결	$\gamma = \rho \mu(\delta) \mu(\delta')$	두 endpoint 사이 closing edge 요구
공통 moment ratio	$\mu(d) = \sum_i w_i^{d+1} / \sum_i w_i^d$	앞 슬라이드의 기대 degree 합
계산 순서	시작 edge $\rightarrow$ DFS tree edges $\rightarrow$ non-tree edges	각 추가 직전 degree로 갱신

Case 1은 이웃 확장, Case 2는 이미 선택한 두 endpoint의 추가 edge 제약이다

같은 pattern도 graph model이 plan 순위를 바꾼다

논문 Figure 2의 두 partial pattern은 PR에서 예상 크기 차이가 크게 난다

β	$ R(P, \text{left-deep}) $	$ R(P, \text{bushy}) $	단위
2.1	67,618.5	230.2	billion
2.5	1,993.0	16.3	billion
2.9	83.4	1.5	billion

PR 추정에서는 bushy partial이 훨씬 작아 ER 기반 left-deep 선택과 달라진다

PR 비용은 estimate이지 observed guarantee가 아니다

모델은 plan 순위를 위한 proxy이며 실제 match·runtime·skew를 완전히 결정하지 않는다

- 입력은 실제 adjacency가 아니라 expected degree sequence와 power-law 가정이다
- 동일 PR-model 계산에서 conflict freedom을 완화한 count는 상한이 된다
- 같은 degree라도 triangle·community에 따라 relation 크기가 달라진다
- MapReduce용 M+S+R cost는 다른 dataflow engine에서 항을 조정해야 한다
- 따라서 선택된 plan의 실제 우수성은 dataset별 실험으로 따로 확인한다

SEED의 optimal plan은 정의된 cost model과 search space 안에서의 optimum이다

Bushy plan은 connected edge-subpattern DP로 찾는다

작은 partial pattern의 최적 비용을 저장하고 큰 pattern의 모든 connected split을 비교한다

01	Enumerate states	P 의 connected edge-subpattern P_a
↓		
02	Base case	P_a 가 join unit이면 join cost 0
↓		
03	Split	P_a 를 connected P_a^l 과 P_a^r 로 분할
↓		
04	Score	두 subplan cost와 두 input 처리 cost 합산
↓		
05	Backtrack	최소 split을 따라 execution tree 복원

Plan state는 data tuple이 아니라 pattern edge subset이므로 작은 query에 적합하다

DP는 subplan과 현재 join input을 함께 센다

부분 pattern P_a 의 최적 비용은 가능한 connected split 중 최소 합으로 정의한다

항	뜻	왜 필요한가
C(E left)	왼쪽 partial을 준비하는 비용	이전 join chain 포함
T(P left)	현재 join에서 왼쪽 입력 처리	Read-shuffle 또는 unit 생성
C(E right)	오른쪽 partial을 준비하는 비용	Bushy child도 독립 계산
T(P right)	현재 join에서 오른쪽 입력 처리	두 입력 volume 모두 반영
Minimum	모든 connected split 비교	Best split과 child를 저장

공간 $O(2^m)$, 시간 $O(3^m)$ 이므로 m 이 작은 pattern query를 전제로 한다

Overlap은 edge를 미리 검사해 후보를 줄인다

같은 vertex 집합에 constraint edge 하나를 추가하면 match 수는 늘지 않는다

구성	Relation	Match 수 관계	Plan 효과
P	기본 partial pattern	$ R(P) $	넓은 후보
P^+	P에 기존 vertex 사이 edge 추가	$ R(P^+) \leq R(P) $	Join 전에 후보 감소 가능
Overlap unit	다른 unit과 edge 일부 공유	Constraint 중복 적용	Intermediate 축소 가능

Edge 중복은 결과 중복이 아니라 earlier filtering을 위한 constraint 중복이다

All-or-nothing은 overlap search를 $O(3^n)$ 으로 되돌린다

두 join side의 cut edge를 전부 추가하거나 하나도 추가하지 않는 두 경우만 비교한다

01	Find cut nodes	두 side가 공유하되 redundant하지 않은 vertex
↓		
02	Find cut edges	Cut node들 사이의 pattern edge
↓		
03	None	Right side에 cut edge를 추가하지 않음
↓		
04	All	모든 cut edge를 right side에 추가
↓		
05	Choose	두 후보 중 더 낮은 estimated cost 선택

같은 order 보장은 두 조건 아래의 asymptotic 결과이며 exact optimum 주장이 아니다

큰 data clique는 정답 조합만으로도 폭발한다

uk graph의 943-node clique 하나는 5-clique 조합 약 6조 개를 포함한다

표현	크기	의미
개별 5-clique matches	$C(943,5) \approx 6 \times 10^{12}$	모든 5개 vertex 조합을 tuple로 저장
Clique vertex set	943 nodes	같은 조합 family를 compact하게 표현
Compression target	Intermediate relation	Join에 필요한 부분만 나중에 확장

압축 이득은 data graph에 큰 clique가 실제로 있을 때 특히 크다

Clique compression은 사전 탐색에서 시작한다

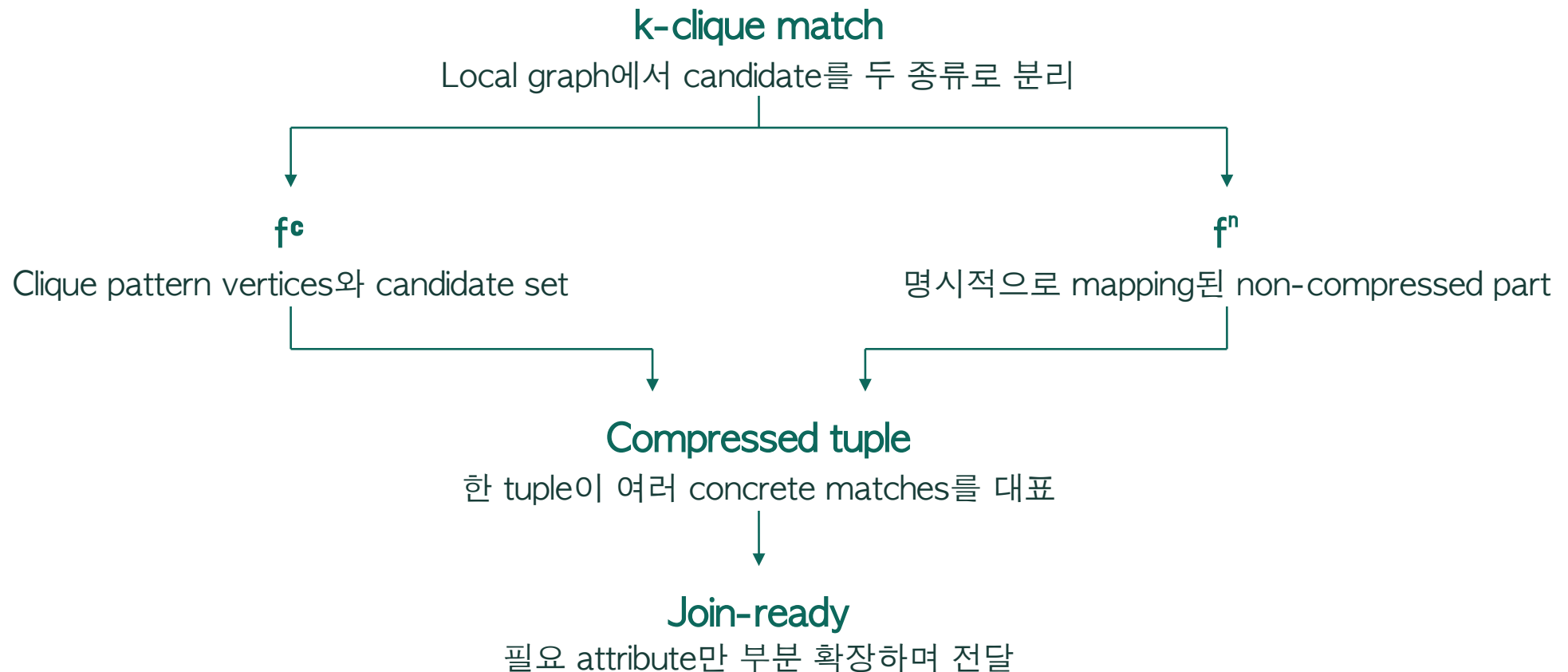
Node-disjoint large clique를 greedy하게 찾고 cluster machine마다 membership index를 둔다

- | | | |
|----|----------------|----------------------------------|
| 01 | Pick vertex | 남은 graph에서 최대 degree vertex 선택 |
| ↓ | | |
| 02 | Maximal clique | 그 vertex를 포함하는 maximal clique 계산 |
| ↓ | | |
| 03 | Threshold | 크기가 기준보다 크면 set S에 보관 |
| ↓ | | |
| 04 | Remove nodes | 선택 clique vertex를 graph에서 제거 |
| ↓ | | |
| 05 | Index $M(u)$ | 각 machine에서 clique membership 조회 |

논문 구현은 모든 clique가 아니라 node-disjoint large clique set을 사용한다

Compressed match는 두 part를 분리한다

같은 large clique에서 자유롭게 고를 vertex와 이미 지정된 mapping을 따로 보관한다



Partial expansion은 join key에 필요한 부분만 펼친다

Compressed candidate 전체를 즉시 조합하지 않고 현재 join attribute만 concrete하게 만든다

- | | | |
|----|-----------------|---------------------------------------|
| 01 | Read compressed | f^c candidate set과 f^n mapping 입력 |
| ↓ | | |
| 02 | Find Vjoin | 현재 join이 요구하는 pattern attributes 선택 |
| ↓ | | |
| 03 | Expand subset | f^c 에서 필요한 수만큼 candidate 조합 |
| ↓ | | |
| 04 | Build key | Expanded value와 f^n 의 join image 결합 |
| ↓ | | |
| 05 | Keep remainder | 쓰지 않은 f^c 는 compressed 상태 유지 |

정확한 모든 결과가 필요하면 마지막에는 남은 compressed part를 확장해야 한다

실험은 여섯 real graph에서 수행됐다

Preprocessing과 query processing을 분리해 보고하고, 세 시간 안에 끝나지 않으면 INF다

항목	설정	해석 경계
Cluster	1 master, 기본 10 slaves	최대 14 slaves 사용
Slave	c3.4xlarge, 16 vCPU, 30GB	Machine당 mapper 6, reducer 6
Storage	HDFS 128MB blocks, replication 3	MapReduce I/O 조건
Datasets	6 real graphs, 최대 Friendster	65.61M vertices, 1,806.07M edges
Timeout	3 hours	Timeout·OOM·error를 INF로 표시

결과는 이 hardware·framework·query set 조건에 묶여 있다

SCP와 large clique는 query 전에 만든다

논문 Table 5는 두 시간을 query runtime과 별도로 보고한다

Dataset	ϕ , build T (G)	Clique T (C)	단위
youtube	27	58	seconds
live-journal	54	170	seconds
uk-2002	841	1,270	seconds
friendster	2,331	368	seconds

Query-only speedup과 end-to-end first-query cost를 구분해 읽어야 한다

q5 on youtube에서 bushy plan은 거대한 reduce2를 피했다

같은 query와 data에서 SEED와 SEED-LD의 중간 output과 runtime을 비교한다

알고리즘	reduce2 output	Estimated cost	Runtime
SEED bushy	3.2 million	471.9 million	306 s
SEED-LD	6,110.9 million	31,365.2 million	7,658 s
비율	약 1,910×	약 66×	약 25×

관측된 runtime 차이는 이 q5-youtube 설정의 결과이며 보편 상수 배수가 아니다

보고된 개선은 query와 graph 구조에 따라 달라진다

각 수치는 어떤 variant·dataset·query를 비교했는지 함께 읽어야 한다

조건	보고 결과	직접 연결되는 기법
q7 on eu	SEED 980s, TT·PSgL INF	Clique unit과 bushy plan
q6 on eu	SEED+O 1,003s, SEED INF	Clique compression
q2 on lj	SEED+O가 TT보다 50× 초과 빠름	전체 SEED+O pipeline
q6 on lj	ϕ_2 54s + query 1,347s, TT 6,968s	Preprocessing 포함 비교
q1 on yt·lj	SEED+O와 TT 동일 plan·성능	추가 기법이 항상 이득은 아님

한 order 이상이라는 요약은 benchmark 전체 조건을 제거한 보편 법칙이 아니다

Graph와 cluster scaling 실험도 조건부다

Data size·average degree·slave 수를 각각 바꿔 SEED+O와 TT를 비교했다

실험 축	관찰	제한
Graph size	Friendster 60% 초과에서 TT q2 timeout	Node-induced samples와 q2·q7
Average degree	q7에서 SEED+O가 TT보다 10-19×	Friendster edge samples
Slave count	6→14에서 두 방식 runtime 감소	Communication overhead도 증가
14 slaves	q7 on lj는 SEED+O만 완료	해당 cluster와 3-hour cap

Scale-out은 intermediate explosion을 없애지 않으며 communication과 함께 평가해야 한다

SEED가 제거하지 못한 경계

확장성은 intermediate를 줄이는 설계에서 나오며 모든 graph-pattern의 비용을 보장하지 않는다

- Exact enumeration의 최종 output이 크면 그 출력 비용은 피할 수 없다
- ϕ_2 join unit은 star와 clique이며 임의 motif를 local 보존하지 않는다
- Planner는 pattern edge 수 m 에 대해 time $O(3^m)$, space $O(2^m)$ 이다
- PR 내부 relaxed count의 상한은 실제 graph-runtime의 상한이 아니다
- Clique compression은 large clique가 적은 graph에서 이득이 작을 수 있다
- 실험은 MapReduce·일곱 query·여섯 graph·3-hour cap에 조건부다

SEED는 output lower bound가 아니라 avoidable intermediate data를 공격한다

SEED를 기억하는 다섯 문장

분산 서브그래프 열거의 병목은 machine 수보다 relation 설계에서 먼저 생긴다

- | | | |
|----|----------|---|
| 01 | Explode | High-degree star가 거대한 후보 relation을 만든다 |
| ↓ | | |
| 02 | Preserve | ϕ_2 가 ordered-min 규칙으로 star-clique를 보존한다 |
| ↓ | | |
| 03 | Estimate | PR model로 partial relation 크기를 추정한다 |
| ↓ | | |
| 04 | Plan | Bushy DP와 overlap으로 intermediate를 줄인다 |
| ↓ | | |
| 05 | Compress | Large clique 조합을 필요한 순간까지 펼치지 않는다 |

핵심 질문은 무엇을 병렬화할까가 아니라 무엇을 만들지 않을까다

공개 출처

논문 본문과 출판 record를 공개 URL로 확인할 수 있다

자료	용도	공개 위치
PVLDB paper PDF	정의·algorithm·표·실험의 직접 근거	vldb.org/pvldb/vol10/p217-lai.pdf
DOI record	서지 identity와 출판 record	doi.org/ 10.14778/3021924.3021937

슬라이드의 page locator는 공개 PVLDB PDF의 인쇄 page 217-228을 따른다